

UNIVERSITY POLITEHNICA OF BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT



MASTER'S THESIS

Automated ROS Infrastructure
Orchestration via LLM-Driven Templating

Alfred Andrei Pietraru

Thesis advisor:

Prof. dr. ing. Dan Marius Novischi

BUCHAREST

2018

CONTENTS

1	Introduction	1
2	Problem Analysis	2
2.1	ROS-Based Robotic Systems Overview	2
2.2	ROS Configuration and Orchestration Challenges	3
2.3	Limitations of Manual and Static Approaches	3
3	State of the Art and Related Work	5
3.1	Existing Orchestration Frameworks	5
3.2	Language-Based and AI-Driven Robotics Interfaces	8
4	Proposed Solution	11
4.1	High-Level System Overview	11
4.2	Mission Interpretation Component	11
4.3	Component Resolution	13
4.4	Configuration Context Builder	14
4.5	Launch File Generation	15
5	Next-Semester Activities	16
6	Conclusion	17

ABSTRACT

Modern robotic platforms such as the AntRobot operate within complex software ecosystems based on the Robot Operating System (ROS2) [1] [2] where system functionality is distributed across multiple interconnected nodes responsible for perception, localization, mapping, planning, and control. Although ROS promotes modular design, configuring these nodes remains a largely manual and error-prone process. Even minor changes, such as switching sensors, modifying mission modes, or adjusting parameters, often require extensive manual edits across multiple YAML and launch files.

This repetitive configuration effort limits reproducibility, scalability, and experimentation speed in both research and applied robotics. Existing orchestration solutions, including ROScribe and RobotKube, provide partial automation but continue to rely on static configuration schemes that restrict flexibility and reuse.

This work proposes a hybrid ROS orchestration framework that combines dynamic configuration abstraction with language-driven automation. The approach is built on three main components: templated ROS configurations using Jinja2, enabling reusable and parameterized setup files; a Large Language Model (LLM) capable of translating high-level mission descriptions into structured ROS configurations; and a synthetic data bootstrapping strategy that expands a limited set of real mission examples into a larger corpus of mission–configuration pairs.

1 INTRODUCTION

ROS-based robotic systems are constructed from loosely coupled nodes that communicate via a publisher–subscriber architecture. Each node typically performs a well-defined task, such as sensor data acquisition, state estimation, or motion planning. Although this separation of concerns promotes modularity, it also requires careful coordination of parameters, topic names, frame hierarchies, and execution dependencies.

Recent developments in cloud robotics, containerization, and orchestration frameworks have improved deployment scalability. Tools such as RobotKube [3] and FogROS [4] facilitate distributed execution and infrastructure management. Meanwhile, language-based systems such as ROScribe [5] demonstrate the feasibility of using large language models (LLMs) to generate code and launch structures from natural language instructions. However, existing approaches either focus primarily on infrastructure orchestration or code generation, and they typically lack a structured reasoning layer capable of translating high-level mission intent into coherent configuration artifacts.

At the same time, LLMs have shown strong capabilities in structured output generation, including JSON schema adherence and code synthesis. This creates an opportunity to explore their integration into robotic configuration pipelines while preserving determinism and structural validity in downstream system assembly.

Despite the modular architecture of ROS, configuring a robotic system remains a highly manual and error-prone process. Small changes in mission objectives—such as switching sensors, enabling mapping, or modifying operational modes—often require updates across multiple YAML files and launch scripts. These modifications must remain internally consistent, particularly with respect to frame identifiers, topic mappings, and dependency relationships between nodes. Current orchestration tools do not provide a unified mechanism to transform high-level semantic descriptions of robotic tasks into fully consistent ROS deployments. As a result, robotic system configuration remains tightly coupled to domain expertise and manual engineering effort. The absence of a structured pipeline connecting mission intent to executable configuration limits reproducibility, slows experimentation, and increases the likelihood of integration errors. The problem addressed in this work is how can natural-language mission descriptions be systematically translated into valid, coherent, and executable ROS configurations in a deterministic and reproducible manner.

2 PROBLEM ANALYSIS

2.1 ROS-Based Robotic Systems Overview

Nowadays robotic systems are increasingly built as distributed software architectures composed of multiple interacting components. The Robot Operating System (ROS) has become the standard framework when developing such systems, providing a flexible middleware that enables modularity, reuse, and interoperability across heterogeneous hardware and software platforms. In ROS-based systems, functionality is decomposed into nodes, each responsible of a well-defined task. These nodes communicate through message passing mechanisms using topics, services, and actions, forming a dynamic computation graph. This publisher-subscriber architecture enables developers to construct complex robotic behaviors. Nodes can be developed independently, deployed on different machines, and connected at runtime, allowing robotic systems to scale from single embedded devices to distributed setups.

Additionally, ROS supports a wide ecosystem of third-party packages that implement common algorithms and hardware interfaces, significantly reducing development effort. However, the flexibility offered by ROS also introduces complexity. The overall system behavior emerges not only from the implementation of individual nodes, but also from how they are connected, configured, and parameterized. The ROS computation graph may involve dozens of nodes with intricate dependencies on sensors, coordinate frames, timing assumptions, and runtime parameters. As a result, the correctness and performance of the system depend heavily on the configuration choices made during deployment. More than that, ROS itself intentionally avoids imposing strong constraints on how systems should be structured. While this design choice promotes generality and reusability, it places a significant burden on developers to ensure that node interactions are valid for a given task.

As robotic systems grow in scale and complexity, understanding and managing these interactions becomes challenging, particularly when systems must be adapted to new missions, hardware configurations, or operational environments. Beyond software modularity, ROS-based systems must also operate under real-world constraints that further complicate system design. Robotic applications are often required to integrate heterogeneous sensors, operate under strict timing requirements, and interact with uncertain and dynamic environments. These constraints introduce implicit assumptions into system configurations, such as expected sensor frequencies, coordinate frame hierarchies, and synchronization between perception and control pipelines. While ROS provides the communication primitives needed to express these interactions, it does not enforce correctness or consistency at the system level. As a result, many design decisions are encoded implicitly in configuration files rather than being explicitly

represented or validated. This implicit knowledge becomes increasingly difficult to maintain as systems evolve, particularly when configurations are reused across different platforms or mission scenarios.

2.2 ROS Configuration and Orchestration Challenges

Configuring a ROS-based robotic system involves more than launching executable nodes. Developers must specify a wide range of parameters that govern node behavior, communication interfaces, sensor usage, and algorithmic settings. These configurations are typically expressed through a combination of launch files, parameter files, often in YAML format, and environment-specific settings. Even minor changes, such as replacing a sensor or adjusting an algorithm parameter, may require coordinated updates across multiple configuration files. Orchestration further complicates this process. In distributed deployments, nodes may be executed across multiple machines or containers, requiring careful management of networking, synchronization, and lifecycle events. Ensuring that nodes are launched in the correct order, recover gracefully from failures, and remain consistent across restarts is non-trivial, particularly when systems are deployed in dynamic or resource-constrained environments.

Existing orchestration mechanisms within ROS provide only limited support for these challenges. Launch files primarily describe static execution graphs and offer minimal capabilities for conditional logic or dynamic adaptation. External orchestration frameworks, such as container-based deployment platforms, can manage process lifecycles and resource allocation but operate largely independently of ROS-specific semantics and in most cases such systems are not designed and do not support ROS components. As a result, developers must manually bridge the gap between system-level orchestration and ROS-level configuration. Another key challenge lies in the lack of abstraction over mission intent. ROS configurations describe how components are connected and parameterized, but not why a particular configuration is appropriate for a given task. Mission objectives, such as exploration, mapping, or navigation are implicitly encoded in configuration choices rather than explicitly represented. This makes it difficult to reason about configurations at a higher level or to systematically adapt systems to new tasks without extensive manual intervention.

2.3 Limitations of Manual and Static Approaches

The predominant approach to configuring ROS-based systems remains manual and largely static. Developers typically create configuration files by hand, relying on experience, documentation, and trial-and-error to achieve correct system behavior. While this approach may be manageable for small systems or fixed use cases, it does not scale well as systems grow in complexity or must support multiple operational modes. Manual configuration is inherently error-prone. Parameter mismatches, incompatible topic connections, or incorrect assumptions

about sensor availability can lead to subtle runtime failures that are difficult to diagnose. Moreover, configurations are often tightly coupled to specific hardware setups and mission contexts, reducing their reusability across different scenarios. Adapting an existing system to a new task frequently involves duplicating and modifying large portions of configuration code, increasing maintenance effort and the risk of inconsistencies. Static configurations also limit adaptability. Once a system is launched, its structure and parameters typically remain fixed, even if environmental conditions or task requirements change. While dynamic reconfiguration mechanisms exist, they are rarely used systematically due to their complexity and lack of integration with higher-level reasoning. As a result, robotic systems often lack the ability to adjust their behavior automatically in response to changing mission objectives. These limitations show a fundamental gap between the low-level configuration mechanisms provided by ROS and the high-level reasoning required for flexible, mission-driven robotic systems. Closing this gap requires methods that can reason about system structure, parameters, and dependencies in a more abstract manner, while still producing concrete, executable configurations.

The lack of standardized abstraction mechanisms further increases orchestration challenges in ROS-based systems. Configuration files often mix concerns related to hardware setup, algorithm selection, and mission-specific behavior, resulting in tightly coupled specifications that are difficult to reason about independently. While parameter servers and launch-time arguments provide some degree of flexibility, they do not offer a systematic way to capture higher-level relationships between components. Consequently, orchestration logic is frequently implemented through ad-hoc scripts or duplicated configuration fragments, making systems harder to extend and validate. This fragmentation also affects reproducibility, as small, undocumented changes in configuration can significantly alter system behavior without being reflected in versioned source code.

Another flaw is that understanding the impact of a single parameter change often requires detailed knowledge of multiple interacting components, as well as their underlying assumptions. This makes configuration tasks difficult to delegate or automate and limits the ability of non-expert users to deploy or modify robotic systems safely. Moreover, the absence of explicit validation or reasoning mechanisms means that many configuration errors are only discovered at runtime, potentially leading to system downtime or unsafe behavior. These factors underscore the need for approaches that can externalize configuration knowledge, reduce manual effort, and support systematic reasoning about system structure and behavior.

3 STATE OF THE ART AND RELATED WORK

3.1 Existing Orchestration Frameworks

ROS creates a separation of concerns by using nodes that execute clearly defined tasks and communicate with one another through message passing by implementing a publisher subscriber architecture. In a ROS system, nodes communicate extensively with one another, but they can be abstracted as microservices that are responsible for executing specific actions. These microservices usually interact in complex ways that are difficult to handle manually; for this reason, technologies such as Kubernetes [6] simplify the orchestration of such systems by managing their lifecycles, scaling applications, and handling errors, thereby creating a fault-tolerant and dynamic system. Although Kubernetes is heavily used in the software domain, it lacks orchestration mechanisms tailored to domains such as robotics, as it cannot handle ROS-specific concepts such as node graphs, topic compatibility, or sensor algorithm relationships. RobotKube can improve deployment portability by defining software components that can be containerized using Docker [7] and orchestrated by Kubernetes, and it can assist with managing the lifecycle of ROS nodes. On the other hand, RobotKube cannot provide mission-driven orchestration, as the tool has no inherent understanding of the system's purpose. Orchestration is achieved through interoperability with Kubernetes, while the services handled by RobotKube are built using the ROS framework. RobotKube defines robot specific resources such as ROS components and initializes the RobotKube Operator which is a controller program. The Operator reacts to changes to ROS components and depending on the situation, it can deploy containers, set up networking or restarts components which previously failed.

ROSScribe [5] uses natural language processing to describe system functionalities and generates the initial ROS graph, source code, and launch files. ROSScribe builds an entire ROS workspace using four agents that are invoked in a sequential order. First, the SpecAgent constructs the ROS graph of the project by interpreting user requirements and creates a corresponding development plan. Second, the GenAgent generates the workspace by either retrieving code from public repositories or generating code for the required nodes. Third, the PackAgent analyzes the overall workflow and creates the ROS launch files, along with additional support files such as package.xml, CMakeLists.txt, and README.md. In some cases, the generated ROS workspace may contain errors; in this situation, the SupportAgent is responsible for resolving these issues and ensuring that the system passes static analysis checks. ROSScribe automates code writing, launch file generation, and static code analysis as part of an offline workspace synthesis process. Its operation is limited to the design phase, generating an initial ROS workspace based on user descriptions, without adapting configurations at runtime. Also,

ROSScribe does not handle parameter configuration, forcing users to manually create and populate the YAML files associated with each node. This limitation is significant in ROS-based systems, where parameters are often tightly coupled to sensors, algorithms, and mission requirements, making manual configuration repetitive and error-prone. As a result, ROSScribe cannot infer or adapt configurations based on high-level mission intent, relying instead on static, user-defined parameter settings.

FogROS [4] is an adaptive framework designed to automate the deployment of fog robotics applications. As robotic algorithms become increasingly complex, particularly in areas such as perception and mapping, edge-based systems may struggle to handle the associated computational load under certain conditions. One approach to addressing this limitation is to leverage cloud resources, which can provide scalable computational capabilities for demanding operations, subject to network and latency constraints. FogROS builds on top of the ROS framework and allows users to specify which components of a ROS computation graph should be executed on cloud resources. FogROS does not provide reasoning or decision-making capabilities, and all deployment configurations must be specified manually by the user. As a result, the system cannot infer mission intent or dynamically adapt configurations based on high-level task descriptions. An additional feature of FogROS is its ability to automate container creation for cloud deployment, significantly reducing manual setup effort. The framework also supports flexible networking and pre-deployed nodes, abstracting many aspects of deployment across local devices and cloud environments, while still requiring user-defined configuration choices.

OSRF Nexus [8] is a framework designed to support dependency management and version control within ROS-based software ecosystems. It provides mechanisms for organizing, distributing, and resolving ROS packages and their dependencies across different systems, helping ensure consistency and reproducibility of software environments. Nexus operates at the package and dependency level, allowing developers to manage compatible versions of ROS components without directly modifying application logic. However, OSRF Nexus focuses exclusively on software dependency resolution and does not address system-level configuration or orchestration. It does not generate ROS graphs, launch files, or parameter configurations, nor does it provide mechanisms for adapting configurations based on task requirements or mission context. As a result, while Nexus simplifies dependency management and improves build-time reproducibility, it does not contribute to mission-driven orchestration or dynamic configuration synthesis, relying instead on static, user-defined system setups.

Existing tools for ROS-based system development address specific aspects of configuration, deployment, or dependency management, but remain limited when considered from a mission-driven orchestration perspective. ROSScribe automates the generation of ROS workspaces, code, and launch files at design time, yet produces static artifacts and requires manual parameter configuration, lacking support for runtime adaptation or mission-level reasoning. RobotKube improves deployment portability and lifecycle management through Kubernetes integration, but relies on pre-defined configurations and operates solely at the infrastructure

level, without semantic understanding of robotic tasks. FogROS focuses on deployment-time placement of ROS nodes across edge and cloud resources, assuming pre-existing applications and static configurations, and does not support graph synthesis or dynamic reconfiguration. OSRF Nexus enhances dependency and version management, but does not address configuration generation, orchestration, or task-aware system behavior.

Overall, these approaches improve isolated stages of the ROS workflow while relying on static, user-defined configurations. None provide a reasoning layer capable of translating high-level mission intent into adaptive ROS configurations, highlighting a gap that motivates the proposed research approach.

Table 1: Comparison of Existing ROS-Oriented Tools

Tool	Provided Capabilities	Limitations
ROSScribe	Natural-language-driven generation of ROS workspaces, including source code, ROS graphs, launch files, and basic static analysis using a multi-agent pipeline.	Operates only at design time; generates static artifacts; does not handle parameter configuration, runtime adaptation, or mission-level reasoning.
RobotKube	Containerized deployment and lifecycle management of ROS-based components using Kubernetes; improves portability, scalability, and infrastructure-level fault tolerance.	Relies on pre-defined configurations; lacks semantic understanding of ROS graphs, sensor–algorithm relationships, and mission-driven orchestration.
FogROS	Deployment-time placement of ROS nodes across edge and cloud resources; automated container creation and infrastructure abstraction for fog robotics applications.	Assumes pre-existing ROS applications and static configurations; does not support graph synthesis, parameter abstraction, or runtime mission adaptation.
OSRF Nexus	Dependency and version management for ROS packages; improves reproducibility and consistency of ROS software environments.	Operates solely at the package and dependency level; does not address configuration generation, orchestration, or task-aware system behavior.

3.2 Language-Based and AI-Driven Robotics Interfaces

Despite the fact that the outputs of large language models (LLMs) remain nondeterministic, they can still be integrated into robotic systems in a controlled manner. One example of such integration is ROScribe, which leverages LLM-based workflows to automate the generation of ROS workspaces. Similar to the approach described in the AutoGen [9] framework, ROScribe employs a multi-agent pipeline in which agents exchange intermediate artifacts—such as generated code—and invoke external tools to enrich, validate, or evaluate the produced content. Another relevant work for the current context [10] demonstrates how high-level task intent can be extracted from a single natural language instruction and decomposed into multiple subtasks. In this approach, the language model proposes candidate actions expressed in natural language, while the robot provides perception and actuation capabilities, effectively serving as the interface to the physical environment. The robot is equipped with a set of predefined elementary skills, typically involving both perception and motion, which can be composed to execute more complex tasks. In this framework, the LLM is used to evaluate the suitability of available skills at each step by estimating their contribution to task completion through an affordance-based scoring mechanism. The task is defined using a natural language instruction, while each elementary skill is associated with a short description and execution constraints. A similar abstraction could be applied to ROS-based systems, where nodes are treated as modular components characterized by their code, parameters, dependencies, and functional descriptions. By maintaining a database of such components, an LLM could identify suitable building blocks and decompose a high-level task into lower-level configuration requirements, enabling the mapping of individual components to each subtask. While these approaches focus on task execution and skill selection, they do not address system-level configuration synthesis, which remains the focus of the proposed work.

Large language models have consistently demonstrated the ability to generate and adhere to structured output formats such as JSON [11], which is a crucial requirement for integrating LLMs into software systems. When developing applications that rely on automated reasoning, it is often necessary to convert unstructured inputs, such as natural language descriptions, into a fixed set of structured representations that can be programmatically processed. In this context, LLMs do not act as code executors, but rather as planners and selectors that identify the appropriate tools, components, or configuration options required to fulfill a given task. This capability is particularly relevant for configuration synthesis tasks, where downstream systems depend on well-defined and machine-readable representations, such as parameter sets or orchestration descriptors.

Another foundational contribution to reliable language-based system integration is presented in this paper [12]. This work demonstrates that large language models can be explicitly trained to prioritize instruction adherence over unconstrained text generation through supervised fine-tuning and reinforcement learning from human feedback. By optimizing models to follow user-provided instructions and constraints, the approach significantly improves controllability and

predictability of generated outputs. This capability is essential for applications where natural language serves as a formal interface rather than a conversational medium. In the context of configuration synthesis, instruction-tuned models are better suited to interpret high-level mission descriptions and translate them into constrained, structured representations without deviating from specified requirements. When combined with schema-constrained generation strategies, instruction-following behavior enables the use of LLMs as reliable planners that map intent to machine-readable configuration contexts.

Before modifying model parameters through fine-tuning or reinforcement learning, large language models can be adapted to specific tasks through prompt-based techniques. These approaches aim to improve model behavior by optimizing how tasks, constraints, and output formats are expressed to the model, while keeping the underlying model weights unchanged. Prompt-based adaptation is particularly attractive in early-stage system development due to its low computational cost, reversibility, and rapid iteration cycle. One class of approaches relies on text-based prompt optimization, where prompts remain human-readable and are refined through structured experimentation. Libraries such as DSPy [13] support this paradigm by treating prompts as composable and optimizable programs. DSPy enables the construction of modular prompting pipelines and supports evaluation-driven prompt refinement, allowing developers to improve reliability and schema adherence without retraining the model. This makes text-based prompt optimization suitable for exploring task formulations and enforcing structured outputs in configuration synthesis workflows. A complementary approach involves soft or continuous prompt tuning, where prompts are represented as learned embedding vectors rather than explicit text. Frameworks such as PEFT provide implementations of prompt tuning and prefix tuning techniques, in which a small number of trainable prompt embeddings are optimized while the base model remains frozen. These methods allow task-specific behavior to be encoded numerically, offering greater stability and expressiveness than manual prompting, while remaining significantly more efficient than full model fine-tuning. Together, these approaches provide a progressive path for adapting large language models, enabling reliable task execution through prompt design before committing to more costly model modification techniques.

Recent advances in large language models have demonstrated strong capabilities in code-aware reasoning and structured program generation. Models such as DeepSeek-Coder are explicitly trained on large-scale code corpora and optimized to generate syntactically correct and semantically coherent source code across multiple programming languages. These models have shown improved performance in tasks such as code completion, refactoring, and generation of structured artifacts, making them suitable candidates for applications that require reliable synthesis of machine-readable outputs. The relevance of code-oriented language models extends beyond traditional software development to configuration-centric domains, where structured representations such as JSON or YAML must be generated with high precision. In this context, language models can be leveraged to transform natural language descriptions into configuration templates, parameter sets, or orchestration descriptors that can be further processed by downstream systems. This aligns with recent work demonstrating that

code-specialized models exhibit stronger adherence to formal structure compared to general-purpose language models. However, despite their strengths in program synthesis, models such as DeepSeek-Coder operate without domain-specific knowledge of robotic systems. They lack an inherent understanding of ROS concepts, mission semantics, or the relationships between sensors, algorithms, and execution contexts. As a result, while code-aware language models can reliably generate structured configuration artifacts, they require an external abstraction layer to provide semantic grounding and ensure that generated outputs correspond to valid and meaningful robotic system configurations.

4 PROPOSED SOLUTION

4.1 High-Level System Overview

The proposed system is designed as a modular, multi-stage pipeline that transforms high-level human intent into executable robotic configurations in a structured and reproducible manner. As illustrated in the figure below, the process begins with a user-provided natural language mission description, which is interpreted by a language model operating under a fixed instruction context. This interpretation stage produces a structured JSON representation that captures the essential semantic aspects of the mission, including task type, required capabilities, sensor modalities, and operational constraints. The structured mission representation serves as a stable interface between the language-based reasoning layer and the deterministic configuration synthesis pipeline. Subsequent stages validate and normalize this representation before using it to populate parameterized configuration templates, ultimately generating standard ROS configuration and launch artifacts that can be executed without further language model involvement.

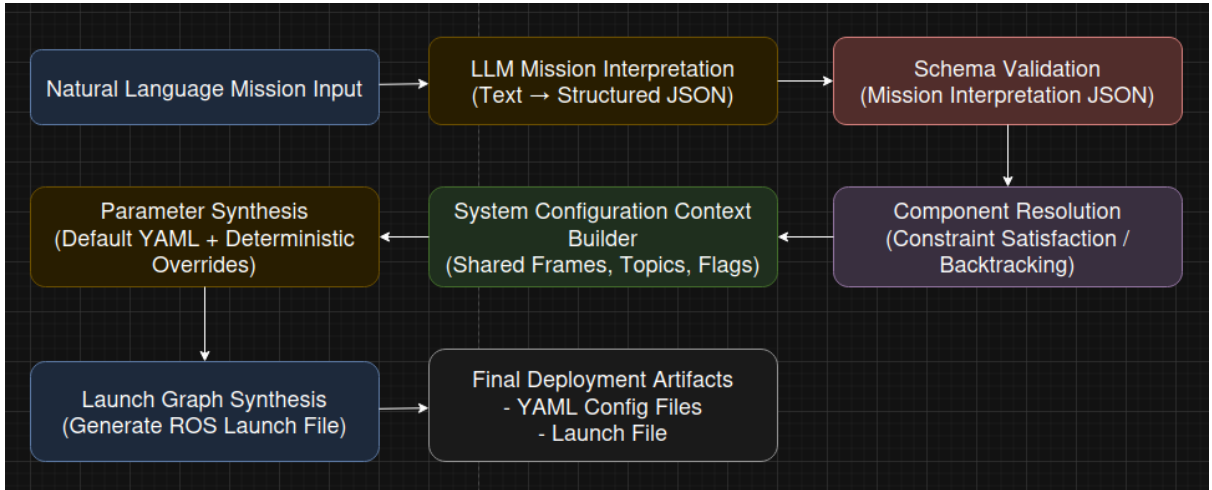


Figure 1: Proposed LLM-Driven ROS Orchestration Architecture

4.2 Mission Interpretation Component

The mission interpretation component represents the first stage of the proposed orchestration framework and serves as the primary interface between human intent and the automated configuration pipeline. Its role is to convert a user-provided natural language mission description into a structured, machine-interpretable representation that captures the essential

characteristics of the intended robotic task. This component is intentionally designed to operate independently of ROS-specific implementation details, focusing instead on extracting abstract mission semantics that can be consumed deterministically by downstream processes. At its core, the mission interpretation process is formulated as a classification task over multiple parameters. The user provides an unstructured textual description of the desired system behavior, such as the type of task to be performed, the operational mode, or the sensors involved. This input is processed by a large language model operating under a fixed system prompt that defines the model’s role, expected behavior, and output format.

The output of this component is a structured JSON object that conforms to a predefined schema. This schema encodes high-level mission attributes, including mission type, required capabilities, sensor modalities, and optional environmental assumptions or operational constraints. By enforcing strict schema adherence and restricting the output space through enumerated values, the system ensures that all generated interpretations are syntactically valid, semantically constrained, and suitable for automatic validation. Free-form text or explanatory output is explicitly disallowed, as the resulting representation is intended to function as data rather than human-readable documentation.

A key design principle of the mission interpretation component is the isolation of nondeterminism. While the language model may internally employ complex reasoning to resolve ambiguities or infer implicit requirements, this nondeterministic behavior is confined exclusively to the interpretation stage. Once a valid JSON representation is produced and validated, all subsequent stages of the pipeline operate deterministically, relying on rule-based logic and templated configuration synthesis. This separation improves reproducibility, debuggability, and system robustness, as errors can be traced either to incorrect interpretation or to downstream processing without ambiguity.

The mission interpretation component is also designed to support progressive model adaptation. The system architecture remains unchanged regardless of whether the language model is used in a zero-shot, few-shot, prompt-optimized, or fine-tuned setting. Improvements in interpretation quality can be achieved by refining prompts, introducing soft prompt tuning, or incorporating additional contextual information, without modifying the interface contract or downstream components. This flexibility enables incremental enhancement of the system while preserving a stable and well-defined mission representation format.

To support the mission interpretation component and enable progressive adaptation of the language model, a structured dataset is constructed consisting of pairs of natural language mission descriptions and their corresponding structured mission representations. Rather than attempting to enumerate all possible robotic missions, the dataset is built around a small set of representative canonical missions that capture common robotic tasks, sensor modalities, and operational modes. For each canonical mission, a gold-standard JSON representation is manually defined according to the mission interpretation schema, ensuring semantic correctness and schema validity.

To improve robustness to linguistic variability, multiple paraphrased mission descriptions are generated for each canonical mission while preserving the same structured output. This synthetic expansion allows the dataset to capture natural variation in user phrasing without introducing ambiguity in the intended system behavior. The resulting dataset supports zero-shot and few-shot evaluation, as well as prompt optimization techniques, while remaining lightweight and incrementally extensible. Larger datasets and more advanced adaptation strategies, such as soft prompt tuning or fine-tuning, are considered as future work and can be incorporated without modifying the system architecture or interpretation interface.

The structured mission representation produced by the interpretation stage is subsequently processed by a validation and normalization component that enforces consistency and system-level constraints. This stage operates in a fully deterministic, rule-based manner and serves to isolate nondeterministic language model behavior from downstream configuration synthesis. Validation verifies the semantic compatibility of mission attributes, such as the coherence between mission type, required capabilities, and sensor modalities, and rejects unsupported or contradictory combinations. Normalization complements this process by injecting predefined default values, making implicit requirements explicit, and standardizing the representation into a canonical form. As a result, semantically equivalent mission descriptions yield identical normalized outputs, ensuring reproducibility and simplifying subsequent processing. The normalized mission context produced by this stage provides a stable and reliable input for configuration synthesis, enabling deterministic generation of ROS configuration artifacts without further reliance on language-based reasoning.

4.3 Component Resolution

The component resolution stage bridges the gap between abstract mission intent and concrete ROS implementations. After the mission interpretation phase produces a validated JSON structure, the system must select a subset of components from a predefined catalog such that the resulting configuration is functionally complete and internally consistent. This stage is entirely deterministic and does not involve further language model reasoning.

The component catalog is structured as a database of declarative entries, each describing a ROS component together with its semantic and compatibility metadata. Each component entry contains: (i) a unique identifier and type (e.g., algorithm, sensor driver, control node), (ii) a list of abstract capabilities it provides, (iii) required capabilities and required sensors, (iv) environment compatibility constraints (e.g., indoor/outdoor), and (v) ROS implementation details such as package name, executable node, and associated launch template. By separating semantic metadata from implementation details, the catalog enables reasoning at the capability level while preserving a direct mapping to concrete ROS artifacts.

Formally, the resolution task can be modeled as a constraint satisfaction problem (CSP). Each component in the catalog is associated with a binary decision variable indicating whether it is

selected. The constraints enforce: (1) capability coverage, meaning that every required mission capability must be provided by at least one selected component; (2) sensor compatibility, ensuring that selected components require only sensors available in the mission specification; (3) dependency closure, such that if a selected component requires another capability, that capability must also be provided within the selected set; and (4) conflict avoidance, preventing mutually incompatible components from being selected simultaneously. A configuration is considered valid only if all constraints are satisfied.

One practical solution strategy is depth-first backtracking with forward constraint propagation. The algorithm incrementally selects components that satisfy required capabilities, while continuously verifying compatibility and dependency constraints. If a partial selection violates any constraint, the algorithm backtracks and explores alternative combinations. Given the limited size of the component catalog in the AntBot scenario, this approach is computationally tractable and guarantees that, if a valid configuration exists, it will be found. Conceptually, the same problem can also be expressed as a Boolean satisfiability (SAT) problem. Each component corresponds to a Boolean variable, and constraints such as capability coverage, dependencies, and mutual exclusions can be encoded as logical clauses. The resolution task then becomes the problem of determining whether there exists a truth assignment that satisfies all clauses. Although industrial SAT solvers are not required due to the small problem scale, this formulation provides theoretical grounding and demonstrates that component selection is a well-defined combinatorial consistency problem rather than a heuristic process.

By formulating component resolution as a constrained selection problem and solving it through deterministic search with constraint propagation, the system ensures that generated configurations are structurally valid, dependency-consistent, and aligned with mission requirements before any ROS configuration files are produced.

4.4 Configuration Context Builder

Following component resolution, the system does not immediately generate individual parameter files for each selected node. Instead, an intermediate configuration context is constructed to ensure global consistency across all components. This step is essential because many parameters in ROS-based systems are not independent. Frame identifiers (e.g., `map`, `odom`, `base_link`), topic names (e.g., `/scan`, `/camera/depth/points`), and time synchronization flags (e.g., `use_sim_time`) must remain consistent across multiple nodes. Generating parameters independently for each component would risk inconsistencies that could break the overall system. To address this, a system-level configuration context is created after component selection. This context aggregates mission information and hardware constraints into a structured representation containing shared parameters such as frame names, topic mappings, sensor activation flags, and execution mode (e.g., `exploration` or `localization-only`). These shared parameters form the common configuration layer of the system.

Each selected component retains a stable default parameter file that reflects algorithm-specific tuning and validated baseline settings. Rather than regenerating full configuration files from scratch, the system applies a deterministic override process. Shared parameters from the global configuration context are injected into the relevant fields of each component's default YAML file, and only a limited set of mission-dependent parameters are modified.

By separating shared system configuration from component-specific defaults, this step guarantees reproducibility, and structural validity. The resulting parameter files are therefore consistent at both the global system level and the individual component level, forming a reliable foundation for subsequent template-based launch file generation.

4.5 Launch File Generation

The final stage of the proposed framework produces a fully executable ROS system configuration derived deterministically from a natural-language mission description. At this point, all semantic reasoning and constraint resolution steps have already been completed. The system now materializes the configuration into concrete deployment artifacts. This output consists of two elements: a generated parameter files for each selected component and a synthesized ROS launch file that instantiates the system graph. Stable, algorithm-specific default YAML configurations are preserved, while mission-dependent values and globally shared system parameters such as frame identifiers, topic names, and execution mode are injected deterministically. This ensures both internal consistency across nodes and preservation of validated tuning parameters. The launch synthesis stage then assembles the executable graph by instantiating each selected node and linking it to its corresponding generated parameter file. The resulting launch file represents the complete operational system, ready to be executed without further manual modification.

5 NEXT-SEMESTER ACTIVITIES

The first semester of this project focused on establishing the conceptual and architectural foundation of the proposed system through a state-of-the-art analysis and formal system design. This phase included a literature review on ROS orchestration frameworks, YAML automation approaches, container-based deployment strategies, and LLM code synthesis methods. Comparative analysis tables were developed to identify the limitations of existing tools and to position the proposed framework within the broader research landscape.

A layered system architecture was designed, clearly defining the data flow between mission interpretation, component resolution, configuration synthesis, and launch generation modules. The mission interpretation schema, component database model, and constraint satisfaction strategy for component selection were formally specified. Particular attention was given to separating semantic interpretation from deterministic configuration synthesis in order to ensure structural validity and reproducibility. The planned activities are as follows:

- Development of a reusable Jinja2-based templating engine for parameterized ROS configuration files
- Define a set of 10–15 representative missions derived from AntRobot’s operational modes, covering scenarios such as SLAM exploration, localization-only navigation, mapping with LiDAR, RGB-D-based mapping, and navigation with obstacle avoidance.
- Generate paraphrased mission descriptions using instruction-tuned models (e.g., Mistral or DeepSeek) in order to expand the dataset to approximately 100 mission variants. These missions will be paired with their corresponding validated configuration outputs and stored in a structured JSONL format.
- Implement an inference pipeline using the Hugging Face Transformers framework to convert natural-language mission descriptions into structured JSON representations
- Experiment with prompt engineering strategies (zero-shot and few-shot prompting) to enforce strict schema adherence before considering any model fine-tuning techniques.
- Connect the validated JSON outputs to the existing component resolution and templating system to enable complete end-to-end configuration generation.
- Validate generated configurations in simulation and, where feasible, on the physical AntRobot platform to ensure structural correctness and reproducibility.

By the end of the second semester, the expected outcomes include a synthetic mission configuration dataset, an LLM-assisted mission interpretation module, and a fully integrated prototype capable of transforming natural-language instructions into executable ROS deployments. This phase will complete the transition from a deterministic configuration framework to a hybrid semantic-to-execution orchestration system.

6 CONCLUSION

This report presented the design and formalization of a structured framework for mission-driven ROS configuration, integrating semantic interpretation with deterministic system synthesis. The work addressed the limitations of manual and static configuration processes in ROS-based robotic systems, where small changes in mission objectives often require extensive modifications across parameter files and launch scripts.

The proposed architecture separates concerns into clearly defined layers: natural-language mission interpretation, schema validation, component resolution through constraint satisfaction, system-level configuration context construction, parameter synthesis using and launch graph generation. By combining language models for high-level semantic abstraction with deterministic algorithms for structural validation and configuration materialization, the framework preserves both flexibility and reproducibility.

A key contribution of this semester is the formal modeling of component selection as a constraint satisfaction problem, ensuring capability coverage, dependency consistency, and sensor compatibility without relying on probabilistic reasoning. Additionally, the introduction of a system-level configuration context guarantees parameter coherence across nodes, addressing one of the most common sources of integration errors in modular robotic systems.

BIBLIOGRAPHY

- [1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "Ros: an open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.
- [3] RWTH Aachen University, "Robotkube: Kubernetes-based robotic infrastructure management," <https://github.com/ika-rwth-aachen/robotkube>, 2025.
- [4] M. G. Y. Kumar, A. Agrawal, S. Levine, and K. Goldberg, "Fogros2: An adaptive framework for automating fog robotics deployment," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023.
- [5] RoboCoach Technologies, "Roscribe: Natural language interface for ros task configuration," <https://github.com/RoboCoachTechnologies/ROSScribe>, 2025.
- [6] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at google with borg," in *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*. ACM, 2015.
- [7] D. Merkel, "Docker: Lightweight linux containers for consistent development and deployment," in *Linux Journal*, 2014.
- [8] Open Source Robotics Foundation, "Osrif nexus: Ros dependency and version management," <https://github.com/osrf/nexus>, 2025.
- [9] S. S. et al., "Autogen: Enabling next-gen llm applications via multi-agent conversation framework," *arXiv preprint arXiv:2308.08155*, 2023.
- [10] S. Y. F. et al., "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.
- [11] B. Agarwal, I. Joshi, and V. Rojkova, "Think inside the json: Reinforcement strategy for strict llm schema adherence," *arXiv preprint arXiv:2502.14905*, 2025.
- [12] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray et al., "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

- [13] O. Khattab, C. Potts, and M. Zaharia, “Dspy: Compiling declarative language model calls into self-improving pipelines,” *arXiv preprint arXiv:2310.03714*, 2023.
- [14] Pallets Projects, “Jinja2 documentation,” <https://jinja.palletsprojects.com>, 2025.
- [15] A. M. et al., “Mistral 7b: A 7-billion-parameter language model engineered for superior performance and efficiency,” *arXiv preprint arXiv:2310.06825*, 2023.
- [16] D. G. et al., “Deepseek-coder: When the large language model meets programming,” *arXiv preprint arXiv:2401.14196*, 2024.
- [17] —, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [18] J. Xie, S. Syu, and H.-Y. Lee, “Exploring the impact of instruction-tuning on llm’s susceptibility to hallucinations,” in *Proceedings of ACL*, 2025.
- [19] T. B. B. et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [20] J. W. et al., “Chain-of-thought prompting elicits reasoning in large language models,” *arXiv preprint arXiv:2201.11903*, 2022.
- [21] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.